Introduction à la data science et à la visualisation

Informatique — Licence 2 (L2)

Cours INF-L2-25

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Découvrir la démarche complète de la science des données avec Python : poser une question, collecter et nettoyer des données réelles avec pandas, les explorer par les statistiques descriptives, les visualiser honnêtement avec matplotlib, puis entraîner et évaluer un premier modèle d'apprentissage supervisé avec scikit-learn.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Qu'est-ce que la science des données
• Leçon 2 — Collecte et nettoyage des données
• Leçon 3 — Exploration et statistiques descriptives
• Leçon 4 — Visualisation efficace des données
• Leçon 5 — Introduction à l'apprentissage automatique supervisé
• Leçon 6 — Mini-projet : analyser un jeu de données réel
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Chaque jour, les téléphones, les écoles, les commerces et les administrations produisent des quantités énormes de données : notes d'étudiants, ventes, trajets, mesures de capteurs, inscriptions en ligne. Ces données brutes ne valent rien par elles-mêmes ; elles deviennent précieuses quand on sait les transformer en informations, puis en décisions. C'est exactement le métier de la science des données (data science) : un mélange de statistiques, de programmation et de bon sens métier, que ce cours te fait pratiquer de bout en bout avec Python.

Tu disposes déjà des deux fondations nécessaires : la programmation Python (cours INF-L2-02) et les statistiques (cours INF-L2-13). Ce cours les assemble dans une démarche unique, celle que suivent les professionnels : poser une question précise, collecter les données, les nettoyer — c'est souvent la plus longue étape —, les explorer, les visualiser, puis éventuellement entraîner un modèle d'apprentissage automatique et évaluer honnêtement ce qu'il vaut. Tout le cours suit un même fil rouge : un petit jeu de données fictif de 26 étudiants, avec ses défauts volontaires, que nous nettoierons et analyserons ensemble.

Un avertissement avant de commencer : la data science ne consiste pas à obtenir des chiffres impressionnants, mais des chiffres justes. Un modèle annoncé à « 99 % de précision » sur des données mal nettoyées ou évalué sur ses propres données d'entraînement ne vaut rien. Ce cours insiste donc autant sur la rigueur — séparer entraînement et test, signaler les limites, ne jamais confondre corrélation et cause — que sur la technique. C'est cette honnêteté qui distingue un data scientist d'un simple utilisateur de logiciels.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Définir la science des données et décrire les étapes d'un projet d'analyse, de la question à la conclusion.
- Charger un fichier CSV avec pandas et diagnostiquer ses défauts : doublons, valeurs manquantes, types incorrects, valeurs impossibles.
- Nettoyer un jeu de données en justifiant chaque décision (suppression, imputation par la médiane, correction de format, écrêtage).
- Explorer des données avec describe(), les corrélations et les regroupements (groupby), en distinguant corrélation et causalité.
- Choisir le graphique adapté à chaque question et repérer les visualisations trompeuses (axe tronqué, camembert illisible, 3D décorative).
- Entraîner une régression linéaire et un arbre de décision avec scikit-learn, en séparant rigoureusement entraînement et test.
- Évaluer un modèle avec les bonnes métriques (R^2, erreur absolue moyenne, précision) et expliquer le sur-apprentissage.

Leçon 1 — Qu'est-ce que la science des données

La science des données est la discipline qui extrait des connaissances et des réponses à partir de données. Elle combine trois ingrédients : les statistiques, pour résumer et raisonner juste sur les chiffres ; l'informatique, pour manipuler des volumes qu'aucun humain ne traiterait à la main ; et la connaissance du domaine, car une moyenne ne dit rien tant qu'on ne sait pas ce qu'elle mesure. Une donnée brute (« 14 ») devient une information quand on sait qu'il s'agit d'une note sur 20 en algorithmique, et une connaissance quand on comprend ce qu'elle implique pour la suite.

Un projet de data science suit presque toujours le même cycle en cinq étapes. Un : formuler une question précise et mesurable (« le temps de travail personnel influence-t-il la réussite ? »), car sans question, on se noie. Deux : collecter les données pertinentes, depuis des fichiers, des enquêtes, des bases ou des capteurs. Trois : nettoyer, car les données réelles sont toujours imparfaites. Quatre : explorer et visualiser pour comprendre la structure et repérer les relations. Cinq : modéliser si nécessaire, puis communiquer les conclusions avec leurs limites. Ce cours suit exactement ce cycle, leçon après leçon.

Les métiers sont variés : l'analyste de données décrit et visualise (tableaux de bord, rapports) ; le data scientist modélise et prédit ; l'ingénieur machine learning met les modèles en production. Tous partagent la même boîte à outils en Python : pandas pour manipuler les tableaux de données, matplotlib pour les graphiques, et scikit-learn pour l'apprentissage automatique — les trois bibliothèques de ce cours. Enfin, travailler sur des données impose une éthique : respecter la vie privée, anonymiser les personnes, et signaler quand un échantillon trop petit ou biaisé interdit de conclure.

L'essentiel
La data science = statistiques + programmation + connaissance du domaine, appliquées selon un cycle fixe : question → collecte → nettoyage → exploration/visualisation → modèle et communication. La rigueur (limites, honnêteté) fait partie du métier.

Exemple — Premier contact avec pandas : les ventes d'une boutique

```
import pandas as pd

# Ventes d'une petite boutique (en milliers de FCFA)
ventes = pd.DataFrame({
"jour": ["Lun", "Mar", "Mer", "Jeu", "Ven", "Sam"],
"ventes": [45, 52, 38, 61, 74, 90],
"clients": [32, 40, 25, 44, 58, 71],
})
print(ventes.shape) # (6, 3) : 6 lignes, 3 colonnes
print(ventes["ventes"].mean()) # 60.0 milliers de FCFA par jour
print(ventes.loc[ventes["ventes"].idxmax()]) # meilleur jour : Sam
```

À retenir

- Une donnée ne devient utile qu'interprétée : valeur + contexte + question = information exploitable.
- Le cycle d'un projet : question, collecte, nettoyage, exploration, modèle, communication — toujours dans cet ordre.
- pandas manipule les données, matplotlib les dessine, scikit-learn les modélise : ce sont les trois outils du cours.

Exercice

Pour chaque situation, dis s'il s'agit de DÉCRIRE (statistiques sur le passé), PRÉDIRE (estimer l'avenir ou une valeur inconnue) ou CLASSER (ranger dans des catégories), puis propose une question de data science précise et mesurable : a) Le secrétariat veut connaître la moyenne générale de la classe au semestre 1. b) Une boutique veut estimer ses ventes du mois prochain à partir des 12 derniers mois. c) Un service de messagerie veut ranger automatiquement les messages en « spam » ou « normal ». d) L'université veut savoir si les étudiants qui assistent à plus de 80 % des cours réussissent mieux.

✔ Corrigé
a) DÉCRIRE : c'est un résumé du passé ; question mesurable : « quelle est la moyenne générale (et la médiane) des notes du semestre 1 ? ». b) PRÉDIRE : on estime une valeur future inconnue ; question : « à combien s'élèveront les ventes du mois prochain, avec quelle marge d'erreur ? ». c) CLASSER : chaque message reçoit une étiquette parmi des catégories connues ; question : « peut-on ranger automatiquement les messages en spam / normal avec moins de 2 % d'erreurs ? ». d) DÉCRIRE puis comparer deux groupes : question : « quelle est la différence de taux de réussite entre les étudiants dont l'assiduité dépasse 80 % et les autres ? ». Remarque de méthode : dans les quatre cas, la question fixe dès le départ les données à collecter et la métrique qui dira si la réponse est bonne — c'est pour cela qu'on la formule avant de toucher aux données.

Leçon 2 — Collecte et nettoyage des données

Les données d'un projet proviennent de sources variées : fichiers CSV ou Excel exportés d'un logiciel, formulaires et enquêtes, bases de données interrogées en SQL, journaux de serveurs, capteurs, ou interfaces de programmation (API). Le format CSV — un tableau en texte brut, une ligne par enregistrement, des colonnes séparées par des virgules — reste le plus courant pour échanger des données. Mais quelle que soit la source, un constat s'impose : les données réelles n'arrivent jamais propres. La collecte n'est donc que la moitié du travail ; l'autre moitié s'appelle le nettoyage.

Cinq défauts reviennent sans cesse. Les doublons : le même enregistrement saisi deux fois, qui fausse les comptages. Les valeurs manquantes : une case vide là où l'information n'a pas été relevée. Les types incorrects : un nombre stocké comme texte (« 13,5 » avec une virgule, « 7 500 » avec un espace) que Python refuse d'additionner. Les valeurs impossibles : une assiduité de 104 %, un âge négatif, qui signalent une erreur de saisie. Et les formats incohérents : des dates tantôt « 2026-01-06 », tantôt « 06/01/2026 ». Chaque défaut non traité se transforme en résultat faux, parfois sans aucun message d'erreur : c'est le danger principal.

Nettoyer, c'est décider, défaut par défaut, en justifiant chaque choix. Un doublon exact se supprime. Une valeur manquante sur la variable qu'on veut prédire rend la ligne inutilisable ; sur une variable d'entrée, on peut l'imputer, par exemple par la médiane, robuste et simple. Une valeur impossible peut être ramenée dans les bornes (écrêtage) ou traitée comme manquante. Deux règles d'or : ne jamais modifier le fichier original — on travaille sur une copie et on sauvegarde le résultat sous un nouveau nom — et garder la trace de chaque correction, car une analyse doit pouvoir être refaite et vérifiée par quelqu'un d'autre.

L'essentiel
Données réelles = données sales : doublons, manquants, mauvais types, valeurs impossibles, formats mélangés. On nettoie sur une copie, défaut par défaut, en documentant chaque décision : supprimer, imputer par la médiane, corriger le format ou écrêter.

Exemple — Nettoyer le relevé des étudiants, de 26 lignes brutes à 23 lignes fiables

```
import io
import numpy as np
import pandas as pd

CSV_BRUT = """prenom,age,sexe,heures,assiduite,note
Awa,19,F,12,92,15.5
Moussa,20,M,8,85,12.0
Fatoumata,19,F,10,90,14.0
Ibrahim,21,M,5,70,9.5
Mariam,20,F,14,95,16.5
Sekou,22,M,3,60,7.0
Kadidiatou,19,F,9,88,13.0
Oumar,20,M,11,80,13.5
Aminata,21,F,6,75,10.5
Cheick,20,M,7,82,11.0
Rokia,19,F,13,93,15.0
Abdoulaye,22,M,4,65,8.5
Nene,20,F,8,86,12.5
Salimata,19,F,15,97,17.0
Youssouf,23,M,2,55,6.0
Assa,20,F,7,84,11.5
Lassine,21,M,9,81,12.5
Hawa,19,F,11,89,14.5
Modibo,20,M,6,72,10.0
Djene,19,F,10,87,"13,5"
Amadou,21,M,8,104,11.5
Korotoumou,20,F,9,83,
Souleymane,21,M,,77,10.5
Adama,20,M,-3,70,8.0
Mariam,20,F,14,95,16.5
Abdoulaye,22,M,4,65,8.5
"""
df = pd.read_csv(io.StringIO(CSV_BRUT))
print("Forme brute :", df.shape) # (26, 6)
print("Doublons :", df.duplicated().sum()) # 2
df = df.drop_duplicates()
df["note"] = pd.to_numeric(df["note"].astype(str).str.replace(",", "."),
errors="coerce")
df = df.dropna(subset=["note"]) # note absente : ligne inutilisable
df.loc[df["heures"] < 0, "heures"] = np.nan # -3 h : impossible
df["heures"] = df["heures"].fillna(df["heures"].median()) # médiane : 8.0
df["assiduite"] = df["assiduite"].clip(0, 100) # 104 % ramené à 100 %
df.to_csv("etudiants_propre.csv", index=False)
print("Forme propre :", df.shape) # (23, 6)
print("Manquants restants :", int(df.isna().sum().sum())) # 0
```

À retenir

• Diagnostiquer avant de corriger : shape, dtypes, duplicated(), isna() donnent la liste exacte des défauts.

• Variable à prédire manquante → ligne supprimée ; variable d'entrée manquante → imputation possible (médiane).

• On ne touche jamais au fichier brut : le nettoyage produit un nouveau fichier, avec des décisions documentées.

Exercice

Un relevé de températures quotidiennes (en °C) contient ces six lignes : (lundi, 31), (mardi, 33), (mardi, 33), (mercredi, vide), (jeudi, 312), (vendredi, « 29,5 »). Pour chaque défaut présent, nomme-le et propose le traitement adapté, en justifiant. Précise quelles lignes ton relevé nettoyé contiendra, et ce que tu fais de la valeur 312.

✔ Corrigé

Quatre défauts distincts. 1) Doublon exact : la ligne (mardi, 33) apparaît deux fois → on la supprime, sinon mardi compterait double dans les moyennes. 2) Valeur manquante : mercredi n'a pas de température ; on ne peut pas l'inventer — on supprime la ligne des calculs ou on la marque manquante (on n'impute que si une méthode justifiée existe, par exemple la moyenne des jours voisins, à condition de le documenter). 3) Valeur impossible : 312 °C n'existe pas à Bamako ; c'est presque sûrement 31,2 ou 31 saisi sans virgule, mais on ne devine pas — on traite la valeur comme manquante (ou on la corrige uniquement si le registre d'origine permet de vérifier). 4) Type/format : « 29,5 » est un texte avec virgule décimale → on remplace la virgule par un point et on convertit en nombre, ce qui donne 29,5. Relevé nettoyé : lundi 31, mardi 33, vendredi 29,5, plus mercredi et jeudi marqués manquants. Leçon générale : trois des quatre traitements sont sûrs ; la valeur 312 montre le cas où l'honnêteté impose de déclarer une donnée perdue plutôt que d'inventer un chiffre.

Leçon 3 — Exploration et statistiques descriptives

Avant de modéliser quoi que ce soit, on explore : c'est l'analyse exploratoire des données. La première étape consiste à regarder la forme du tableau (combien de lignes et de colonnes), les types, puis un résumé statistique de chaque variable numérique avec describe() : l'effectif (count), la moyenne (mean), l'écart-type (std), le minimum, les quartiles et le maximum. Ces quelques nombres racontent déjà beaucoup : l'étendue des valeurs, leur centre, leur dispersion, et parfois des anomalies — un maximum absurde trahit souvent une erreur de saisie que le nettoyage n'a pas vue.

Tu as vu ces notions en statistiques (INF-L2-13) ; ici, elles deviennent des réflexes de lecture. La moyenne résume le centre mais elle est sensible aux valeurs extrêmes : une seule note de 20 parmi des 8 tire la moyenne vers le haut. La médiane — la valeur du milieu une fois les données triées — résiste à ces extrêmes : si moyenne et médiane sont proches, la distribution est plutôt symétrique ; si elles s'écartent, elle est asymétrique. L'écart-type mesure la dispersion autour de la moyenne. Deux autres outils dominent l'exploration : la corrélation, qui quantifie le lien linéaire entre deux variables numériques (de −1 à +1), et le regroupement (groupby), qui calcule un résumé par catégorie, par exemple la note moyenne par sexe ou par tranche d'heures.

Appliquons cela à notre relevé nettoyé de 23 étudiants. La note moyenne est de 11,89 sur 20, la médiane de 12,0 et l'écart-type de 2,92 : la classe fictive est donc plutôt homogène, centrée juste au-dessus de la moyenne de passage. Le taux de réussite

(note ≥ 10) est de 78,3 %. Deux corrélations ressortent : 0,95 entre les heures d'étude hebdomadaires et la note, et 0,89 entre l'assiduité et la note — des liens forts. Attention toutefois : une corrélation n'est pas une cause. Il est possible que travailler plus améliore la note, mais aussi que les étudiants déjà à l'aise déclarent plus d'heures, ou qu'une troisième variable (la motivation) explique les deux. Et avec 23 étudiants d'une seule classe fictive, ces chiffres décrivent cet échantillon, rien de plus.

L'essentiel
Explorer = describe() + corrélations + groupby avant tout modèle. Moyenne et médiane se comparent (symétrie), l'écart-type mesure la dispersion, la corrélation mesure un lien — jamais une preuve de cause.

Exemple — Explorer le relevé nettoyé : describe, corrélations et moyennes par groupe

```
import pandas as pd

df = pd.read_csv("etudiants_propre.csv") # produit à la leçon 2
print(df[["heures", "assiduite", "note"]].describe().round(2))
print("Corrélation heures / note :", round(df["heures"].corr(df["note"]), 2)) # 0.95
print("Corrélation assiduité / note :", round(df["assiduite"].corr(df["note"]), 2)) #
0.89
print(df.groupby("sexe")["note"].mean().round(2)) # F : 13.95 ; M : 10.00
df["admis"] = df["note"] >= 10
print("Taux de réussite :", round(df["admis"].mean() * 100, 1), "%") # 78.3 %
```

À retenir

- describe() donne en un coup d'œil : effectif, moyenne, écart-type, min, quartiles, max — le premier réflexe d'exploration.
- Moyenne ≈ médiane : distribution symétrique ; moyenne nettement au-dessus de la médiane : quelques grandes valeurs tirent vers le haut.
- Une corrélation de 0,95 est un lien très fort, mais corrélation n'est pas causalité ; et un petit échantillon ne se généralise pas.

Exercice

La commande describe() appliquée aux prix (en FCFA) d'un catalogue de téléphones donne : effectif 40, moyenne 84 000, écart-type 61 000, min 15 000, premier quartile 45 000, médiane 68 000, troisième quartile 105 000, max 390 000. Réponds avec ces seuls chiffres : 1) La distribution est-elle plutôt symétrique ou tirée vers les prix élevés ? Justifie avec deux indicateurs. 2) Quelle est l'étendue des prix ? 3) Le prix maximal te paraît-il plausible pour un téléphone du catalogue, ou suspect ? 4) Pour annoncer un « prix typique » aux clients, vaut-il mieux citer la moyenne ou la médiane ?

✔ Corrigé
1) Elle est tirée vers les prix élevés : la moyenne (84 000) est nettement supérieure à la médiane (68 000), signe que quelques modèles très chers tirent la moyenne vers le haut ; de plus, la distance médiane → max (322 000) est bien plus grande que min → médiane (53 000). 2) Étendue = max − min = 390 000 − 15 000 = 375 000 FCFA. 3) 390 000 FCFA est possible pour un modèle haut de gamme, donc plausible en soi ; mais il vaut la peine de vérifier la ligne correspondante : à ce niveau d'écart avec le troisième quartile (105 000), une erreur de saisie (un zéro de trop, par exemple 39 000 devenu 390 000) reste une hypothèse à contrôler avant de conclure. 4) La médiane : elle représente le prix du modèle « du milieu », insensible aux quelques téléphones très chers qui gonflent la moyenne. C'est le réflexe général : distribution asymétrique → médiane ; distribution symétrique → moyenne et médiane se rejoignent.

Leçon 4 — Visualisation efficace des données

Un bon graphique transmet en quelques secondes ce qu'un tableau de chiffres met des minutes à raconter : une tendance, un écart entre groupes, une valeur aberrante. Mais un graphique n'est jamais neutre : le même jeu de données peut éclairer ou tromper selon les choix de représentation. La première compétence est donc de choisir le graphique adapté à la question. Pour comparer des catégories : des barres. Pour voir la répartition d'une variable numérique : un histogramme. Pour examiner le lien entre deux variables numériques : un nuage de points. Pour suivre une évolution dans le temps : une courbe. Choisir autre chose, c'est répondre à côté de la question.

Les mauvais usages sont partout. Le camembert devient illisible dès cinq ou six parts, et l'œil compare mal les angles : des barres classées feraient mieux. La 3D décorative déforme les surfaces et les volumes sans rien ajouter. Les doubles axes avec des échelles indépendantes fabriquent des croisements spectaculaires qui ne signifient rien. Et surtout, l'axe des ordonnées tronqué : si les barres commencent à 10 au lieu de 0, une différence de 11,9 contre 15,3 — déjà réelle — prend l'allure d'un écart du simple au triple, parce que notre œil juge les barres à leur hauteur visible. Tronquer un axe de barres, c'est mentir avec des chiffres vrais.

Concrètement, avec matplotlib, chaque graphique du cours suit la même hygiène : un titre qui énonce le message, des axes nommés avec leurs unités, une échelle complète pour les barres (ici, ylim(0, 20) car les notes sont sur 20), et une seule idée par figure. Sur notre relevé, le graphique en barres des notes moyennes par tranche d'heures — 7,75 pour 0 à 5 h, 11,58 pour 6 à 10 h, 15,33 pour 11 h et plus — montre d'un coup d'œil la progression régulière déjà suggérée par la corrélation de 0,95. Le nuage de points heures/note la confirme autrement : chaque point est un étudiant, et l'alignement général se voit immédiatement, avec ses quelques points écartés qui méritent qu'on s'y intéresse.

L'essentiel
À chaque question son graphique : barres pour comparer, histogramme pour répartir, nuage de points pour relier, courbe pour suivre le temps. Titre parlant, axes nommés avec unités, barres qui partent de zéro : c'est l'hygiène minimale d'une visualisation honnête.

Exemple — Deux graphiques honnêtes sur le relevé des étudiants

```
import pandas as pd
import matplotlib
matplotlib.use("Agg") # génération sans écran (serveur, script)
import matplotlib.pyplot as plt

df = pd.read_csv("etudiants_propre.csv")
df["tranche"] = pd.cut(df["heures"], bins=[0, 5, 10, 20],
labels=["0-5 h", "6-10 h", "11 h et +"])
moy = df.groupby("tranche", observed=True)["note"].mean()
print(moy.round(2)) # 7.75 ; 11.58 ; 15.33
plt.figure(figsize=(6, 4))
moy.plot(kind="bar", color="#0b3d91")
plt.ylabel("Note moyenne / 20")
plt.xlabel("Heures d'étude par semaine")
plt.title("Note moyenne selon le temps d'étude")
plt.ylim(0, 20) # axe complet : les barres partent de zéro
plt.tight_layout()
plt.savefig("graphique_barres.png", dpi=120)
plt.figure(figsize=(6, 4))
plt.scatter(df["heures"], df["note"], color="#0b3d91")
plt.xlabel("Heures d'étude / semaine")
plt.ylabel("Note / 20")
plt.title("Chaque point = un étudiant")
plt.tight_layout()
plt.savefig("graphique_nuage.png", dpi=120)
```

À retenir

- Comparer des catégories → barres ; répartir une variable → histogramme ; relier deux variables → nuage de points ; suivre le temps → courbe.
- Un axe de barres qui ne part pas de zéro exagère visuellement les écarts : c'est la tromperie graphique la plus fréquente.
- Camembert au-delà de quelques parts et 3D décorative : à éviter, ils rendent les comparaisons impossibles.

Exercice

Un rapport présente les notes moyennes de deux classes sous forme de barres : la classe A affiche 12,4 et la classe B 11,9, mais l'axe vertical commence à 11 et se termine à 13. 1) Pourquoi ce graphique est-il trompeur alors que les deux chiffres sont exacts ? 2) Quelle est la vraie différence entre les deux classes, en points et en pourcentage relatif ? 3) Redécris le graphique corrigé (type, échelle, titre). 4) Pour comparer plutôt la répartition des notes dans chaque classe (y a-t-il beaucoup d'élèves

faibles ou forts ?), quel graphique vaudrait-il mieux utiliser ?

✔ Corrigé
1) Parce que l'œil compare la hauteur visible des barres : avec un axe de 11 à 13, la barre de A mesure 1,4 unité visible et celle de B 0,9 — A paraît plus de 1,5 fois « plus haute », alors que l'écart réel est minime. Les chiffres sont vrais, l'impression est fausse. 2) Différence réelle : 12,4 − 11,9 = 0,5 point, soit environ 4 % de plus pour A (0,5 / 11,9 ≈ 0,042) — un écart faible, presque négligeable. 3) Graphique corrigé : barres verticales avec un axe de 0 à 20 (l'échelle complète des notes), titre explicite du type « Note moyenne par classe (sur 20) », valeurs écrites au-dessus des barres : l'écart de 0,5 point y apparaît pour ce qu'il est, modeste. 4) Pour la répartition, un histogramme par classe (ou deux histogrammes superposés/comparés) : il montre combien d'élèves se situent dans chaque intervalle de notes, ce qu'une moyenne seule masque — deux classes peuvent avoir la même moyenne avec des répartitions très différentes.

Leçon 5 — Introduction à l'apprentissage automatique supervisé

L'apprentissage automatique (machine learning) consiste à laisser un programme ajuster un modèle à partir d'exemples, plutôt que de lui écrire des règles à la main. En apprentissage SUPERVISÉ, chaque exemple d'entraînement est étiqueté : on connaît la bonne réponse, et le modèle apprend à la retrouver à partir des variables d'entrée, notées X, pour pouvoir ensuite l'estimer sur de nouveaux cas. Si la réponse à prédire est un nombre (une note, un prix, une quantité), on parle de RÉGRESSION ; si c'est une catégorie (admis ou ajourné, spam ou normal), on parle de CLASSIFICATION. Dans ce cours, nous utilisons deux modèles simples de scikit-learn : la régression linéaire, qui ajuste une droite (ou un plan) aux données, et l'arbre de décision, qui enchaîne des questions du type « heures ≥ 6 ? » jusqu'à une réponse.

La règle d'or du supervisé est la séparation entraînement / test. Si l'on évalue un modèle sur les exemples qui lui ont servi à apprendre, on mesure sa mémoire, pas sa capacité à généraliser : c'est comme corriger un examen avec les réponses sous les yeux. On réserve donc une partie des données — ici 25 %, soit 6 étudiants sur 23 — que le modèle ne voit jamais pendant l'entraînement, et c'est sur elles qu'on le juge. Pour la régression, deux métriques dominent : le R^2, part de la variance expliquée (1 = parfait, 0 = aussi bon que prédire toujours la moyenne), et l'erreur absolue moyenne (MAE), lisible dans l'unité du problème (« en moyenne, le modèle se trompe de tant de points »). Pour la classification, la précision (accuracy) compte la proportion de bonnes réponses — à condition de regarder aussi la composition du test, comme le montrera notre exemple.

Sur notre relevé, la régression linéaire qui prédit la note à partir des heures et de l'assiduité obtient un R^2 de 0,89 sur le test et une erreur moyenne de 0,57 point : honorable pour un si petit jeu de données. Ses coefficients se lisent concrètement : environ +0,61 point de note par heure d'étude hebdomadaire supplémentaire, et +0,07 point par point d'assiduité, les autres variables restant égales. Le danger symétrique s'appelle le SUR-APPRENTISSAGE (overfitting) : un modèle trop flexible

épouse le bruit des données d'entraînement au lieu de la tendance, brille en entraînement et déçoit en test. L'exercice de cette leçon le montre avec un arbre de régression illimité : R^2 de 1,00 en entraînement — il a pratiquement mémorisé chaque étudiant, avec 16 feuilles pour 17 exemples — mais seulement 0,79 en test, moins bien que la simple droite. Enfin, prudence sur la classification : notre arbre (profondeur limitée à 3) affiche une précision de test de 1,00, mais les 6 étudiants du test sont tous admis — un modèle paresseux répondant toujours « admis » ferait le même score. Une métrique ne vaut que par le test qui l'a produite.

L'essentiel
Supervisé = apprendre à partir d'exemples étiquetés : régression pour un nombre, classification pour une catégorie. On entraîne sur une partie des données, on évalue sur une partie JAMAIS vue (R^2 et MAE pour la régression, précision pour la classification). Un modèle trop flexible sur-apprend : parfait en entraînement, médiocre en test.

Exemple — Régression linéaire et arbre de décision avec scikit-learn

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import r2_score, mean_absolute_error, accuracy_score

df = pd.read_csv("etudiants_propre.csv")
X = df[["heures", "assiduite"]] # variables d'entrée
y_reg = df["note"] # cible de la régression
y_clf = (df["note"] >= 10).astype(int) # cible de la classification
X_tr, X_te, yr_tr, yr_te, yc_tr, yc_te = train_test_split(
X, y_reg, y_clf, test_size=0.25, random_state=42)
print("Tailles train / test :", len(X_tr), "/", len(X_te)) # 17 / 6
reg = LinearRegression().fit(X_tr, yr_tr)
pred = reg.predict(X_te)
print("R2 sur le test :", round(r2_score(yr_te, pred), 2)) # 0.89
print("Erreur moyenne (points) :", round(mean_absolute_error(yr_te, pred), 2)) # 0.57
print("Coefficients [heures, assiduité] :", reg.coef_.round(2)) # [0.61 0.07]
arbre = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_tr, yc_tr)
print("Précision arbre sur le test :", round(accuracy_score(yc_te,
arbre.predict(X_te)), 2)) # 1.00 (voir la leçon !)
```

À retenir

- Régression = prédire un nombre ; classification = prédire une catégorie ; dans les deux cas, X sont les entrées et y la cible étiquetée.
- Jamais d'évaluation sur les données d'entraînement : le test doit contenir des exemples que le modèle n'a jamais vus.
- Sur-apprentissage : écart marqué entre performance d'entraînement et de test ; remèdes : modèle plus simple, profondeur limitée, plus de données.

Exercice

Reprends le découpage de l'exemple (train_test_split, test_size=0.25, random_state=42) et entraîne cette fois un arbre de RÉGRESSION sans aucune limite de profondeur : DecisionTreeRegressor(random_state=42) sur les mêmes X et y_reg. Calcule son R² sur l'entraînement puis sur le test, ainsi que son nombre de feuilles (get_n_leaves()). Compare avec la régression linéaire de l'exemple et conclus : quel modèle généralise le mieux, et quel phénomène les chiffres révèlent-ils ?

✔ Corrigé
Résultats obtenus avec ce code exact : l'arbre de régression illimité atteint un R² de 1,00 sur l'entraînement, mais seulement 0,79 sur le test (erreur moyenne 0,75 point), avec 16 feuilles pour 17 exemples d'entraînement. La régression linéaire, elle, affiche 0,93 en entraînement et 0,89 en test. Diagnostic : sur-apprentissage. Avec presque une feuille par étudiant, l'arbre a mémorisé les réponses au lieu d'apprendre la tendance ; sa perfection en entraînement est donc un signal d'alarme, pas un succès. Le modèle le plus simple généralise mieux ici. Les parades à connaître : limiter la profondeur de l'arbre (avec max_depth=3, le même arbre de régression tombe à 0,98 en entraînement et 0,69 en test — limiter ne garantit pas de gagner, cela change le compromis), collecter davantage de données, et toujours comparer plusieurs modèles sur le même test. Retenir surtout ceci : un écart train/test important est la signature du sur-apprentissage, quelle que soit la famille de modèle.

Leçon 6 — Mini-projet : analyser un jeu de données réel

Cette dernière leçon assemble tout le cours dans la démarche complète d'un mini-projet, tel que tu le rendrais dans un rapport. Un mini-projet tient en cinq livrables : une question précise ; une description des données (source, taille, défauts trouvés et corrections faites) ; l'exploration chiffrée et au moins un graphique honnête ; le cas échéant, un modèle entraîné et évalué selon le protocole entraînement/test ; et une conclusion qui répond à la question, cite ses preuves et déclare ses limites. C'est ce format — et non la longueur du code — qui fait la qualité d'un travail de data science : chaque affirmation doit pouvoir être rattachée à un chiffre effectivement calculé.

Déroulons la démarche sur notre fil rouge. Question : « le temps d'étude hebdomadaire explique-t-il la note des étudiants ? » Données : le relevé fictif de la leçon 2, 26 lignes brutes ramenées à 23 lignes propres après suppression de 2 doublons, d'une ligne sans note et correction des formats, des valeurs impossibles et des manquants. Exploration : la corrélation heures/note vaut 0,95, et les notes moyennes par tranche progressent régulièrement (7,75 ; 11,58 ; 15,33). Modèle simple à une variable : la droite ajustée a pour pente 0,83 — en moyenne, une heure d'étude hebdomadaire de plus est associée à environ 0,8 point de note de plus dans cet échantillon — et pour ordonnée à l'origine 4,90 ; elle prédit 13,2 pour un étudiant à 10 h et 8,2 pour un étudiant à 4 h. Sur l'ensemble des 23 étudiants, son R² est de 0,90 (à ne pas confondre avec le R² de test de la leçon précédente : évaluer sur toutes les données, y compris celles d'entraînement, donne toujours une image plus flatteuse).

Conclusion, rédigée comme elle devrait l'être : dans cet échantillon, le temps d'étude est fortement associé à la note, et la relation est régulière dans les trois tranches

d'heures. Limites obligatoires : l'échantillon ne compte que 23 étudiants fictifs d'une seule classe — aucune généralisation à « les étudiants » n'est permise ; l'assiduité est elle aussi corrélée à la note (0,89), donc une partie de l'effet attribué aux heures peut venir d'elle ; et corrélation n'est pas causalité — il faudrait un suivi réel, avec des mesures répétées, pour affirmer que faire travailler davantage améliorerait les notes. Cette honnêteté n'affaiblit pas le projet : elle le rend crédible. Un mini-projet réussi finit toujours par « ce qu'il faudrait pour aller plus loin » : ici, un vrai relevé anonymisé de plusieurs classes, relevé de la même façon pour tous.

L'essentiel
Un mini-projet = question, données documentées (défauts et corrections), exploration chiffrée, graphique honnête, modèle évalué sur un test, conclusion avec limites déclarées. La crédibilité vient des chiffres réellement calculés et des limites avouées, pas des promesses.

Exemple — Le mini-projet en un seul script : de la question à la prédiction

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Question : le temps d'étude explique-t-il la note ?
df = pd.read_csv("etudiants_propre.csv") # 23 lignes, nettoyé à la leçon 2
corr = df["heures"].corr(df["note"])
modele = LinearRegression().fit(df[["heures"]], df["note"])
print("Corrélation heures / note :", round(corr, 2)) # 0.95
print("Pente :", round(modele.coef_[0], 2)) # 0.83 point par heure
print("Ordonnée à l'origine :", round(modele.intercept_, 2)) # 4.90
p10 = modele.predict(pd.DataFrame({"heures": [10]}))[0]
p4 = modele.predict(pd.DataFrame({"heures": [4]}))[0]
print("Prédiction pour 10 h/sem :", round(float(p10), 1)) # 13.2
print("Prédiction pour 4 h/sem :", round(float(p4), 1)) # 8.2
print("R2 sur tout l'échantillon :", round(modele.score(df[["heures"]], df["note"]),
2)) # 0.90
```

À retenir

- Structure d'un rapport d'analyse : question, données (avec leurs défauts), exploration, modèle évalué, conclusion limitée et honnête.
- Chaque nombre cité dans la conclusion doit avoir été calculé par ton code — jamais estimé « à l'œil » ni promis à l'avance.
- Faire la distinction entre R^2 sur toutes les données (flatteur) et R^2 sur un test jamais vu (crédible).

Exercice

Rédige, en cinq à huit lignes, la conclusion du mini-projet de cette leçon, comme si elle fermait un rapport rendu à un enseignant. Elle doit contenir : la réponse à la question posée, au moins deux chiffres-preuves tirés des leçons (corrélation, pente ou moyennes par tranche), les trois limites du travail (taille et nature de l'échantillon, variable confondante, corrélation contre causalité), et une piste concrète pour aller

plus loin.

✔ Corrigé
Exemple de conclusion attendue : « Dans notre échantillon de 23 étudiants (données fictives d'exercice, une seule classe), le temps d'étude hebdomadaire est fortement associé à la note : la corrélation atteint 0,95 et les notes moyennes passent de 7,75 (0 à 5 h) à 15,33 (11 h et plus). Le modèle linéaire simple estime environ +0,83 point par heure hebdomadaire supplémentaire. Trois limites s'imposent : l'échantillon est petit et fictif, donc non généralisable ; l'assiduité, elle-même corrélée à la note (0,89), peut expliquer une partie de l'association ; enfin, une corrélation ne prouve pas que travailler plus cause de meilleures notes. Pour aller plus loin, il faudrait un relevé réel anonymisé sur plusieurs classes, mesuré de manière homogène, et un modèle évalué sur un jeu test plus conséquent. » Toute conclusion qui reprend ces éléments chiffrés et ces limites, même formulée autrement, est correcte ; une conclusion qui affirmerait « travailler plus donne de meilleures notes » sans réserve serait fautive.

Travaux dirigés (TD)

TD 1 — Les emprunts de la bibliothèque universitaire

La bibliothèque de l'université a relevé, sur six mois, le nombre de livres empruntés et le nombre de retours en retard : Janvier 320 emprunts / 18 retards ; Février 285 / 15 ; Mars 410 / 22 ; Avril 390 / 19 ; Mai 455 / 25 ; Juin 380 / 17. 1) Construis ce tableau avec pandas et calcule : le total des emprunts, la moyenne et la médiane mensuelles, le mois le plus actif. 2) Calcule le taux global de retards (total des retards divisé par total des emprunts). 3) Quel graphique choisirais-tu pour montrer l'évolution des emprunts sur le semestre ? Et pour comparer emprunts et retards mois par mois ? 4) La bibliothécaire affirme : « en mai, les étudiants sont devenus négligents ». Les chiffres la soutiennent-ils ?

✔ Corrigé du TD 1
1) Total des emprunts : 320 + 285 + 410 + 390 + 455 + 380 = 2 240. Moyenne mensuelle : 2 240 / 6 ≈ 373,3 emprunts. Valeurs triées : 285, 320, 380, 390, 410, 455 → médiane = (380 + 390) / 2 = 385. Le mois le plus actif est mai (455 emprunts). 2) Total des retards : 18 + 15 + 22 + 19 + 25 + 17 = 116. Taux global : 116 / 2 240 ≈ 5,2 %. 3) Évolution dans le temps → une courbe (mois en abscisse, emprunts en ordonnée). Comparaison emprunts/retards par mois → des barres groupées (deux barres par mois), car on compare deux séries sur les mêmes catégories. 4) Non, pas directement : mai compte le plus de retards (25) mais aussi le plus d'emprunts ; son taux de retard vaut 25 / 455 ≈ 5,5 %, à peine au-dessus du taux global de 5,2 % et proche de mars (22 / 410 ≈ 5,4 %). En proportion, la « négligence » n'augmente presque pas : c'est le volume qui gonfle les deux nombres. C'est l'erreur classique consistant à comparer des effectifs bruts au lieu de taux — toujours ramener à la même base avant de conclure.

TD 2 — Nettoyer le relevé de ventes d'une boutique

Une boutique exporte son relevé du début janvier (date, produit, quantité, prix unitaire en FCFA) : (2026-01-05, A, 3, 5000) ; (2026-01-05, A, 3, 5000) ; (2026-01-06, B, 2, « 7

500 ») ; (06/01/2026, B, 1, 7500) ; (2026-01-07, C, −1, 12000) ; (2026-01-08, A, quantité vide, 5000) ; (2026-01-08, C, 4, 12000) ; (2026-01-09, B, 2, 7500). 1) Liste tous les défauts présents et le traitement adapté à chacun. 2) Après nettoyage, combien de lignes de ventes valides reste-t-il ? 3) Calcule le chiffre d'affaires total des ventes valides (montant = quantité × prix) et le détail par produit. 4) Que fais-tu de la ligne à quantité −1, et pourquoi ne faut-il ni la supprimer en silence ni la compter comme une vente ordinaire ?

✔ Corrigé du TD 2
1) Défauts : un doublon exact (la première ligne répétée) → suppression ; un prix en texte avec espace « 7 500 » → retirer l'espace et convertir en nombre (7 500) ; une date au format 06/01/2026 incohérente avec les autres → conversion au format commun AAAA-MM-JJ (2026-01-06) ; une quantité manquante → la ligne ne permet aucun calcul de montant : elle est exclue des ventes et signalée ; une quantité négative (−1) → cas particulier traité à la question 4. 2) Sur 8 lignes brutes : moins le doublon, moins la ligne sans quantité → 6 lignes exploitables, dont 5 ventes positives et 1 retour. 3) Montants : A : 3 × 5 000 = 15 000 ; B : 2 × 7 500 + 1 × 7 500 + 2 × 7 500 = 37 500 ; C : 4 × 12 000 = 48 000. Chiffre d'affaires des ventes valides : 15 000 + 37 500 + 48 000 = 100 500 FCFA. 4) La quantité −1 correspond très vraisemblablement à un retour client (montant −12 000 FCFA). La compter comme une vente gonflerait le chiffre d'affaires ; la supprimer en silence ferait disparaître une information réelle et fausserait la trésorerie. Le bon traitement : la sortir des ventes, la conserver dans une catégorie « retours » documentée, et, si le rapport parle de chiffre d'affaires net, présenter les deux nombres : 100 500 FCFA de ventes et −12 000 FCFA de retours, soit 88 500 FCFA nets.

Travaux pratiques (TP)

TP 1 — Ton premier pipeline complet, de bout en bout

Objectif : reproduire, seul et dans l'ordre, toute la chaîne du cours sur le relevé des étudiants. Étape 1 — Recrée le fichier brut en exécutant le code de la leçon 2 (il produit etudiants_propre.csv ; garde aussi une copie du tableau brut en CSV sous le nom etudiants_brut.csv). Étape 2 — Vérifie le nettoyage : le fichier propre doit contenir 23 lignes et 6 colonnes, sans aucune valeur manquante, et l'assiduité maximale doit valoir 100. Étape 3 — Exploration : affiche describe() des colonnes heures, assiduite et note ; calcule les deux corrélations avec la note et le taux de réussite. Étape 4 — Visualisation : produis les deux graphiques de la leçon 4 (barres par tranche d'heures avec axe de 0 à 20, nuage de points heures/note) et enregistre-les en PNG. Étape 5 — Modèle : avec train_test_split (test_size=0.25, random_state=42), entraîne la régression linéaire et l'arbre de décision de la leçon 5 et affiche leurs métriques de test. Étape 6 — Rédige cinq lignes de conclusion avec les limites du travail, sur le modèle de la leçon 6.

✔ Attendus du TP 1
Valeurs attendues, obtenues en exécutant exactement les codes des leçons (pandas 2.1.4, scikit-learn 1.9.1) : Étape 2 — forme propre (23, 6), 0 valeur manquante, assiduité max = 100. Étape 3 — note : moyenne 11,89, médiane 12,0, écart-type 2,92, min 6,0, max 17,0 ; heures : moyenne 8,43, médiane imputée et observée 8,0 ; corrélations : heures/note 0,95, assiduité/note 0,89 ; taux de réussite 78,3 %. Étape 4 — notes moyennes par tranche : 7,75 (0-5 h, 4 étudiants), 11,58 (6-10 h, 13 étudiants), 15,33 (11 h et +, 6 étudiants) ; les deux fichiers PNG sont créés sans erreur. Étape 5 — découpage 17 entraînement / 6 test ; régression linéaire : R^2 test 0,89, erreur moyenne 0,57 point, coefficients ≈ [0,61 ; 0,07] ; arbre profondeur 3 : précision 1,00 sur un test composé uniquement d'admis (à signaler comme limite !). Étape 6 — la conclusion doit citer au moins la corrélation de 0,95 et les trois limites (échantillon fictif de 23 étudiants, assiduité confondante, corrélation contre causalité). Si tes chiffres diffèrent légèrement, vérifie d'abord random_state et l'ordre des étapes de nettoyage : la médiane d'imputation (8,0) dépend des lignes déjà présentes.

Exercices corrigés supplémentaires

Exercice 1

Choisis le graphique le mieux adapté à chaque situation et justifie en une phrase : a) comparer les effectifs des trois filières (Informatique, BBA, Droit) cette année ; b) voir si les notes d'une classe se concentrent autour de 10 ou s'étalent de 2 à 19 ; c) examiner le lien entre le nombre d'heures de révision et la note obtenue, étudiant par étudiant ; d) suivre le nombre d'inscriptions mois par mois sur deux ans.

✔ Corrigé
a) Barres : on compare quelques catégories distinctes, et des barres classées se lisent d'un coup d'œil (un camembert serait acceptable pour 3 parts, mais les barres restent plus précises). b) Histogramme : il montre la répartition d'une variable numérique en intervalles — concentration, étalement et éventuels trous se voient immédiatement. c) Nuage de points : chaque point est un étudiant, les heures en abscisse et la note en ordonnée ; l'alignement éventuel révèle le lien, et les points isolés signalent les cas particuliers. d) Courbe : le temps en abscisse et une ligne continue mettent en valeur la tendance et les variations saisonnières (rentrées, vacances). Règle générale : comparer → barres ; répartir → histogramme ; relier → nuage de points ; suivre → courbe.

Exercice 2

Voici les notes d'un devoir (sur 20) : 8, 12, 12, 15, 9, 20, 12. 1) Calcule la somme, la moyenne, la médiane, le mode et l'écart-type (celui de pandas, Series.std()). 2) On découvre que le 20 était une erreur de saisie : il fallait lire 2. Recalcule moyenne et médiane. 3) Que démontre ce petit accident sur le choix entre moyenne et médiane ?

✔ Corrigé
1) Somme : 8 + 12 + 12 + 15 + 9 + 20 + 12 = 88. Moyenne : 88 / 7 ≈ 12,57. Série triée : 8, 9, 12, 12, 12, 15, 20 → médiane = 12 (la valeur du milieu, 4e sur 7). Mode = 12 (trois occurrences). Écart-type (pandas, std()) ≈ 3,99. 2) Avec 2 à la place de 20 : somme = 70, moyenne = 10,0 ; série triée : 2, 8, 9, 12, 12, 12, 15 → médiane toujours 12. 3) Une seule valeur erronée a fait chuter la moyenne de 12,57 à 10,0 (−2,57 points), alors que la médiane n'a pas bougé : la moyenne additionne toutes les valeurs, donc chaque extrême la tire ; la médiane ne dépend que de l'ordre, donc elle résiste aux valeurs aberrantes. Conséquence pratique : devant des données pouvant contenir des erreurs ou des extrêmes (prix, revenus, notes saisies à la main), on publie la médiane, ou les deux en expliquant l'écart.

Exercice 3

Un étudiant veut prédire la note finale de ses camarades. Il entraîne un modèle avec les colonnes : heures d'étude, assiduité, et « mention », une colonne qu'il a calculée lui-même à partir de la note (Passable si note < 12, Bien si note < 14, etc.). Son modèle obtient une précision presque parfaite, en entraînement comme sur le test. Il conclut : « mon modèle est excellent ». Explique pourquoi cette conclusion est fausse, quel est le nom de ce piège, et comment refaire l'expérience correctement.

✔ Corrigé
Le piège s'appelle la FUITE DE DONNÉES (data leakage). La colonne « mention » est calculée à partir de la note, c'est-à-dire à partir de la réponse que le modèle doit prédire : en la lui donnant en entrée, on lui souffle la réponse. Au moment de prédire la note d'un nouvel étudiant, cette mention est justement inconnue — elle n'existe qu'après coup. Le modèle est donc inutilisable en conditions réelles, malgré des scores parfaits qui mesurent la fuite, pas la compétence du modèle. Refaire correctement : supprimer toute colonne dérivée de la cible (mention, moyenne générale incluant la note, rang dans la classe, décision du jury…), ne garder que des informations disponibles AVANT la prédiction (heures, assiduité, notes des contrôles précédents), puis réévaluer sur un test jamais vu. Règle à retenir : pour chaque colonne d'entrée, se demander « connaîtrai-je cette valeur au moment où je devrai prédire ? » — si la réponse est non, la colonne est interdite.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. La science des données consiste d'abord à…

   A) Acheter le logiciel le plus puissant du marché

   B) Transformer des données en réponses à des questions précises

   C) Remplacer les statistiques par l'intelligence artificielle

2. Quelle est la première étape d'un projet d'analyse de données ?

A) Formuler une question précise et mesurable

B) Entraîner le modèle le plus complexe possible

C) Tracer tous les graphiques imaginables

3. Dans un tableau pandas, un doublon exact non traité risque surtout de…

A) Faire planter Python à tous les coups

B) Accélérer les calculs

C) Fausser les comptages et les moyennes en comptant deux fois le même cas

4. La note (variable à prédire) est manquante sur une ligne. Le traitement standard est de…

A) Remplacer la note par 0

B) Supprimer cette ligne de l'apprentissage

C) Remplacer la note par la moyenne de la classe

5. Pour une distribution de prix très asymétrique (quelques prix énormes), le « prix typique » le plus honnête est…

A) La moyenne, toujours

B) Le maximum

C) La médiane

6. Une corrélation de 0,95 entre heures d'étude et note permet d'affirmer que…

A) Travailler plus CAUSE de meilleures notes, c'est prouvé

B) Les deux variables varient ensemble de façon forte et régulière dans cet échantillon

C) La note explique 95 % des heures d'étude

7. Pour examiner le lien entre deux variables numériques, étudiant par étudiant, le graphique adapté est…

A) Le camembert

B) Le nuage de points

C) Le diagramme en barres

8. Un graphique en barres dont l'axe vertical commence à 11 au lieu de 0 est problématique parce que…

A) Les couleurs deviennent illisibles

B) Python refuse de le tracer

C) Il exagère visuellement les écarts : l'œil compare les hauteurs visibles, pas les valeurs

9. On sépare les données en entraînement et test parce que…

A) Évaluer sur l'entraînement mesurerait la mémoire du modèle, pas sa capacité à généraliser

B) Cela divise le temps de calcul par deux

C) scikit-learn l'exige pour fonctionner

10. Le sur-apprentissage (overfitting), c'est quand un modèle…

A) Est trop simple pour capter la moindre tendance

B) Épouse le bruit de l'entraînement : excellent dessus, décevant sur de nouvelles données

C) Refuse de s'entraîner par manque de données

Corrigés du QCM

1. Réponse B — La data science combine statistiques, programmation et connaissance du domaine pour répondre à des questions : les outils viennent après la question.

2. Réponse A — Sans question claire, on ne sait ni quelles données collecter, ni quelle métrique jugera la réponse : le cycle commence toujours par la question.

3. Réponse C — Un doublon compte un même enregistrement deux fois : effectifs gonflés, moyennes tirées vers le cas dupliqué — sans aucun message d'erreur.

4. Réponse B — Sans la vraie réponse, la ligne ne peut ni entraîner ni évaluer le modèle ; l'imputation (médiane) se réserve aux variables d'entrée. Mettre 0 fabriquerait un faux échec.

5. Réponse C — La médiane résiste aux valeurs extrêmes ; la moyenne, tirée par les quelques prix énormes, décrirait un « prix typique » que presque aucun produit n'a.

6. Réponse B — La corrélation mesure une association dans les données observées ; la causalité exigerait une autre démarche (et une variable confondante, comme la motivation, reste possible).

7. Réponse B — Chaque point y représente un individu avec ses deux valeurs : l'alignement (ou son absence) révèle le lien, et les points isolés signalent les cas particuliers.

8. Réponse C — Avec un axe tronqué, un écart de 0,5 point peut paraître un écart du simple au double : les chiffres sont vrais, l'impression est fausse.

9. Réponse A — Le test contient des exemples jamais vus pendant l'apprentissage : c'est la seule façon honnête d'estimer la performance sur de nouveaux cas.

10. Réponse B — Signature : un écart marqué entre performance d'entraînement et de test — comme un arbre illimité à $R^2 = 1{,}00$ en entraînement mais 0,79 en test. Remèdes : simplifier le modèle, limiter sa profondeur, ajouter des données.

Résumé — les notions importantes

• La data science suit un cycle fixe : question précise, collecte, nettoyage, exploration et visualisation, modèle éventuel, communication avec les limites.

• Les données réelles sont sales : doublons, valeurs manquantes, mauvais types, valeurs impossibles, formats mélangés — on nettoie sur une copie, en documentant chaque décision.

• Variable cible manquante → ligne supprimée ; variable d'entrée manquante → imputation possible, par exemple par la médiane, robuste aux extrêmes.

• describe(), corrélations et groupby résument un jeu de données avant tout modèle ; moyenne et médiane se comparent, et corrélation n'est jamais causalité.

• À chaque question son graphique : barres, histogramme, nuage de points, courbe ; un axe de barres tronqué est une tromperie, même avec des chiffres exacts.

• Apprentissage supervisé : régression (un nombre) ou classification (une catégorie), toujours avec séparation entraînement / test et des métriques adaptées (R^2, MAE, précision lue avec sa composition de test).

• Le sur-apprentissage guette les modèles trop flexibles : seule la performance sur des données jamais vues compte ; un mini-projet crédible cite des chiffres calculés et déclare ses limites.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Qu'est-ce que la science des données
Une donnée ne devient utile qu'interprétée : valeur + contexte + question = information exploitable. ; Le cycle d'un projet : question, collecte, nettoyage, exploration, modèle, communication — toujours dans cet ordre. ; pandas manipule les données, matplotlib les dessine, scikit-learn les modélise : ce sont les trois outils du cours.

Fiche 2 — Collecte et nettoyage des données
Diagnostiquer avant de corriger : shape, dtypes, duplicated(), isna() donnent la liste exacte des défauts. ; Variable à prédire manquante → ligne supprimée ; variable d'entrée manquante → imputation possible (médiane). ; On ne touche jamais au fichier brut : le nettoyage produit un nouveau fichier, avec des décisions documentées.

Fiche 3 — Exploration et statistiques descriptives
describe() donne en un coup d'œil : effectif, moyenne, écart-type, min, quartiles, max — le premier réflexe d'exploration. ; Moyenne ≈ médiane : distribution symétrique ; moyenne nettement au-dessus de la médiane : quelques grandes valeurs tirent vers le haut. ; Une corrélation de 0,95 est un lien très fort, mais corrélation n'est pas causalité ; et un petit échantillon ne se généralise pas.

Fiche 4 — Visualisation efficace des données
Comparer des catégories → barres ; répartir une variable → histogramme ; relier deux variables → nuage de points ; suivre le temps → courbe. ; Un axe de barres qui ne part pas de zéro exagère visuellement les écarts : c'est la tromperie graphique la plus fréquente. ; Camembert au-delà de quelques parts et 3D décorative : à éviter, ils rendent les comparaisons impossibles.

Fiche 5 — Introduction à l'apprentissage automatique supervisé
Régression = prédire un nombre ; classification = prédire une catégorie ; dans les deux cas, X sont les entrées et y la cible étiquetée. ; Jamais d'évaluation sur les données d'entraînement : le test doit contenir des exemples que le modèle n'a jamais vus. ; Sur-apprentissage : écart marqué entre performance d'entraînement et de test ; remèdes : modèle plus simple, profondeur limitée, plus de données.

Fiche 6 — Mini-projet : analyser un jeu de données réel
Structure d'un rapport d'analyse : question, données (avec leurs défauts), exploration, modèle évalué, conclusion limitée et honnête. ; Chaque nombre cité dans la conclusion doit avoir été calculé par ton code — jamais estimé « à l'œil » ni promis à l'avance. ; Faire la distinction entre R^2 sur toutes les données (flatteur) et R^2 sur un test jamais vu (crédible).

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — C'est quoi la DATA SCIENCE ? (Expliqué simplement en 5 min)

Chaîne : Data-Junior · Langue : Français · Niveau : Débutant
Une présentation claire et sans jargon de la science des données : ce que c'est, comment les entreprises s'en servent pour décider, et les étapes d'un projet, de la collecte au modèle. Parfait avant la leçon 1, pour situer le cours.

https://www.youtube.com/watch?v=35gtyk1zcX0

Leçon 2 — Leçon 11 : Python : supprimer les doublons et corriger les types de données avec Pandas

Chaîne : Wissen corporation · Langue : Français · Niveau : Débutant à intermédiaire
Exactement les gestes de la leçon 2, en vidéo : détecter et supprimer les lignes dupliquées avec drop_duplicates(), puis vérifier et corriger les types des colonnes avec dtypes et info() avant toute analyse.

https://www.youtube.com/watch?v=hpFoNyBhHos

Leçon 3 — Apprendre Python, Numpy, Pandas pour la Data Science [Cours complet Niveau Débutant avec Notebooks]

Chaîne : J.A DATATECH CONSULTING · Langue : Français · Niveau : Débutant
Un cours complet et progressif sur l'usage de pandas pour l'analyse de données : DataFrames, sélections, résumés statistiques et manipulations courantes. À regarder par chapitres, en parallèle des leçons 3 et 6.

https://www.youtube.com/watch?v=LwkWwxg10IU

Leçon 4 — MATPLOTLIB - Les Bases ! (14/30)

Chaîne : Machine Learnia · Langue : Français · Niveau : Débutant à intermédiaire

Les bases de matplotlib par un data scientist : première figure avec plt.plot(), styles utiles, subplots pour afficher plusieurs graphiques et version orientée objet. Le complément direct de la leçon 4.

https://www.youtube.com/watch?v=O_OeWxpnUc0

Leçon 5 — PYTHON SKLEARN: KNN, LinearRegression et SUPERVISED LEARNING (20/30)

Chaîne : Machine Learnia · Langue : Français · Niveau : Intermédiaire

La prise en main de scikit-learn : le principe de l'apprentissage supervisé, le fonctionnement d'un estimateur (fit, predict, score), un exemple de régression linéaire et un de classification. À voir après la leçon 5 pour consolider.

https://www.youtube.com/watch?v=P6kSc3qVph0

Sources et références

- Documentation officielle de pandas (pandas.pydata.org/docs) — manipulation de DataFrames, nettoyage, describe(), groupby : la référence quotidienne de ce cours.
- Documentation officielle de scikit-learn (scikit-learn.org) — guide de l'utilisateur : apprentissage supervisé, séparation entraînement/test et métriques d'évaluation.
- Documentation officielle de Matplotlib (matplotlib.org) — galerie d'exemples de graphiques et guide des bonnes pratiques de visualisation.
- Wes McKinney — « Python for Data Analysis » (O'Reilly, édition en accès libre en ligne) : le livre du créateur de pandas, pour approfondir la préparation des données.
- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.